



UNIVERSITÀ DEGLI STUDI DI MILANO



Generative AI for automated legacy mainframe documentation

Riccardo Sturla

Data Science for Economics
Academic Year 2023–2024

Supervisor: Prof. Alfio Ferrara

Co-supervisor: Prof. Stefano Montanelli

External Co-supervisor: Dr. Enrico Papalini

Overview

- Problem Formulation
- Objectives
- Available Data
- State of the art
- Direct Approach
- Chain Approach
- Multi-agent Approach
- Prompt Engineering
- Evaluation Framework
- Results
- Further Implementations
- Conclusion

Problem formulation

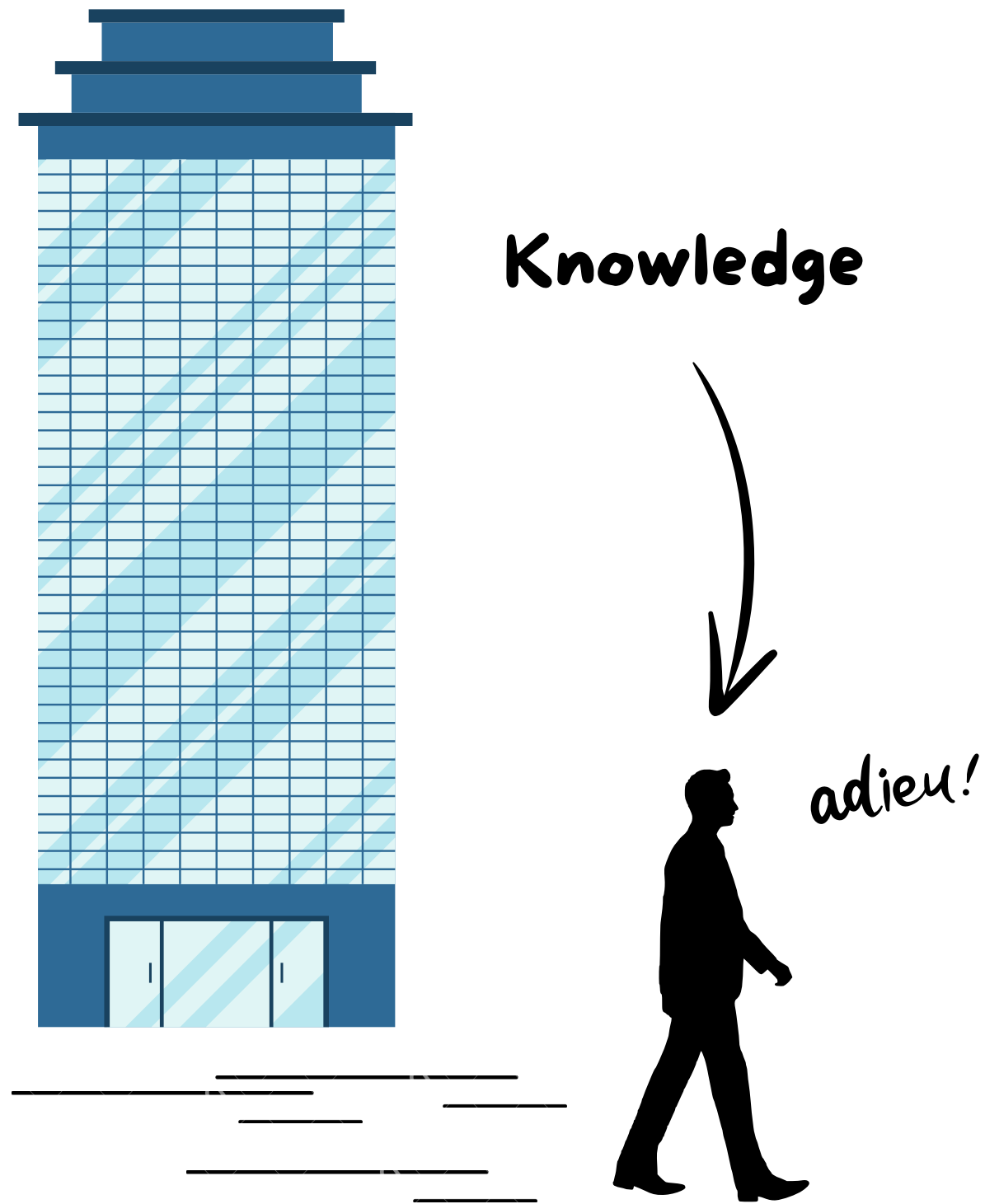


Issues

- Legacy mainframe applications built on **outdated programming languages**.
- These languages are no longer taught in schools and universities.
- **Minimal documentation** for older systems.
- Developers struggle to understand complex, undocumented systems.
- **Retiring developers** create a knowledge gap.

Consequences

- Difficulty **maintaining** and **updating** legacy systems.
- Risky business decisions due to **lack of expertise**.
- Delayed development and troubleshooting.
- Higher operational **costs** and **inefficiency**.
- Industry-wide impact, particularly in banking and insurance.



*"Knowledge is leaving the building and no documentation is left behind.
It seems like this is starting to be a common problem around the world."*

Bill Hinshaw, Founder COBOL Cowboys

Create documentation using Generative AI

Roadmap

- Evaluate the **feasibility** of a **Gen. AI** solution.
- Develop an **internal solution** by testing various approaches to find the best fit for the use case.
- Explore **open-source alternatives**, comparing the quality of the results obtained.
- Develop solid prompts through **prompt engineering** techniques.
- **Evaluation** and **comparison** using a custom framework.

Available Data



COBOL

- Main Objective
- Used in ES **Milan**
- 7000 lines of code
- Longest program: **60k tokens**

Extend the scope of our experiments to **other CSDs** in the group:



EGL

Used in ES **Copenhagen**

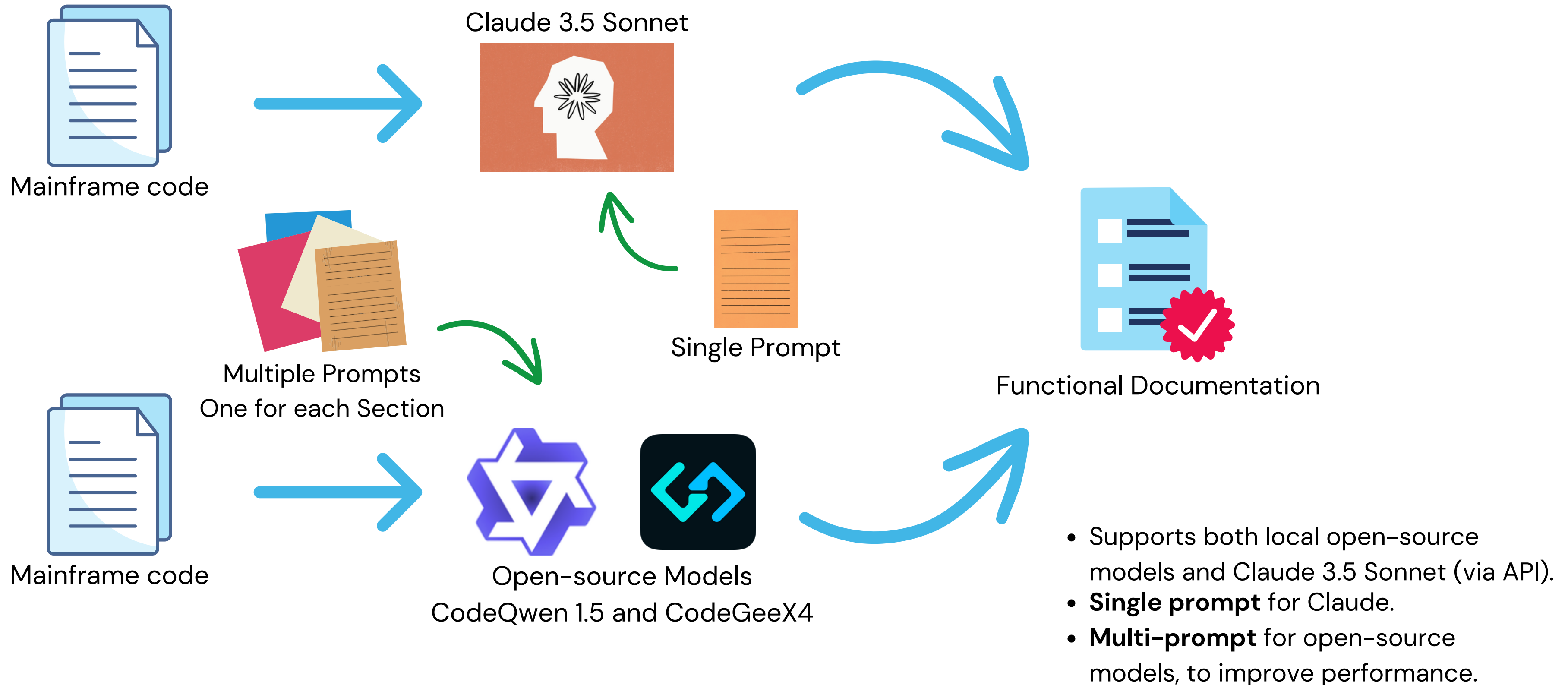


PL/1

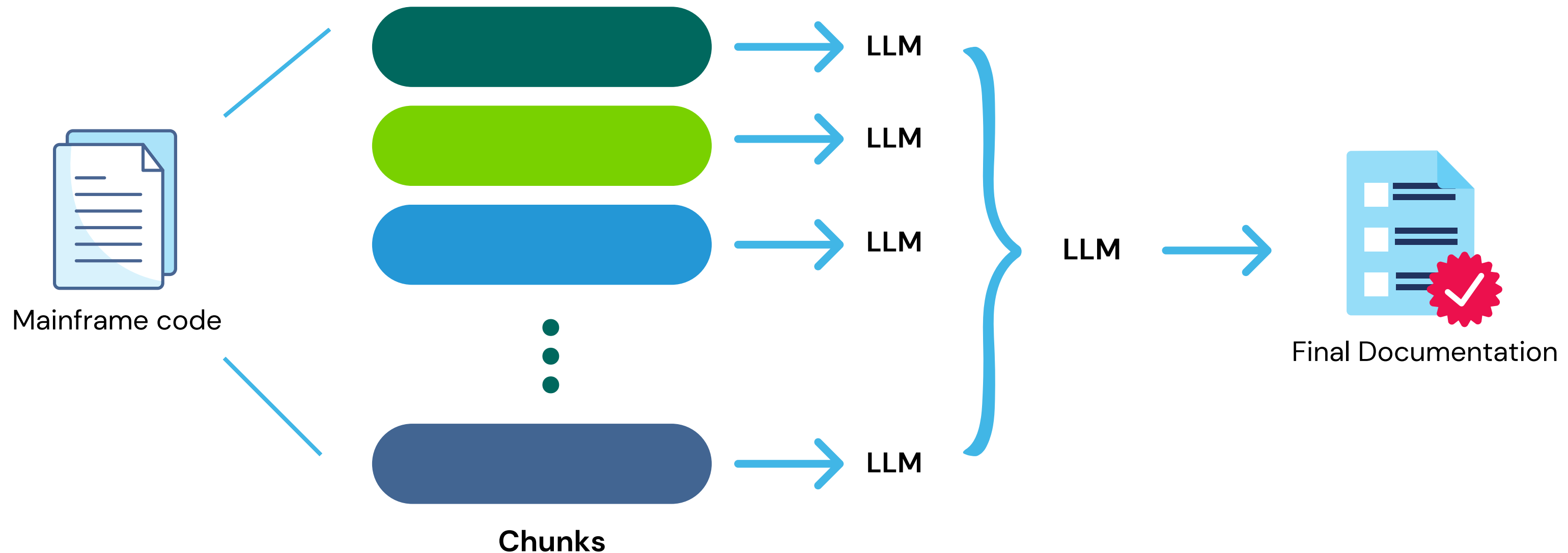
Used in ES **Oslo**



Direct Approach

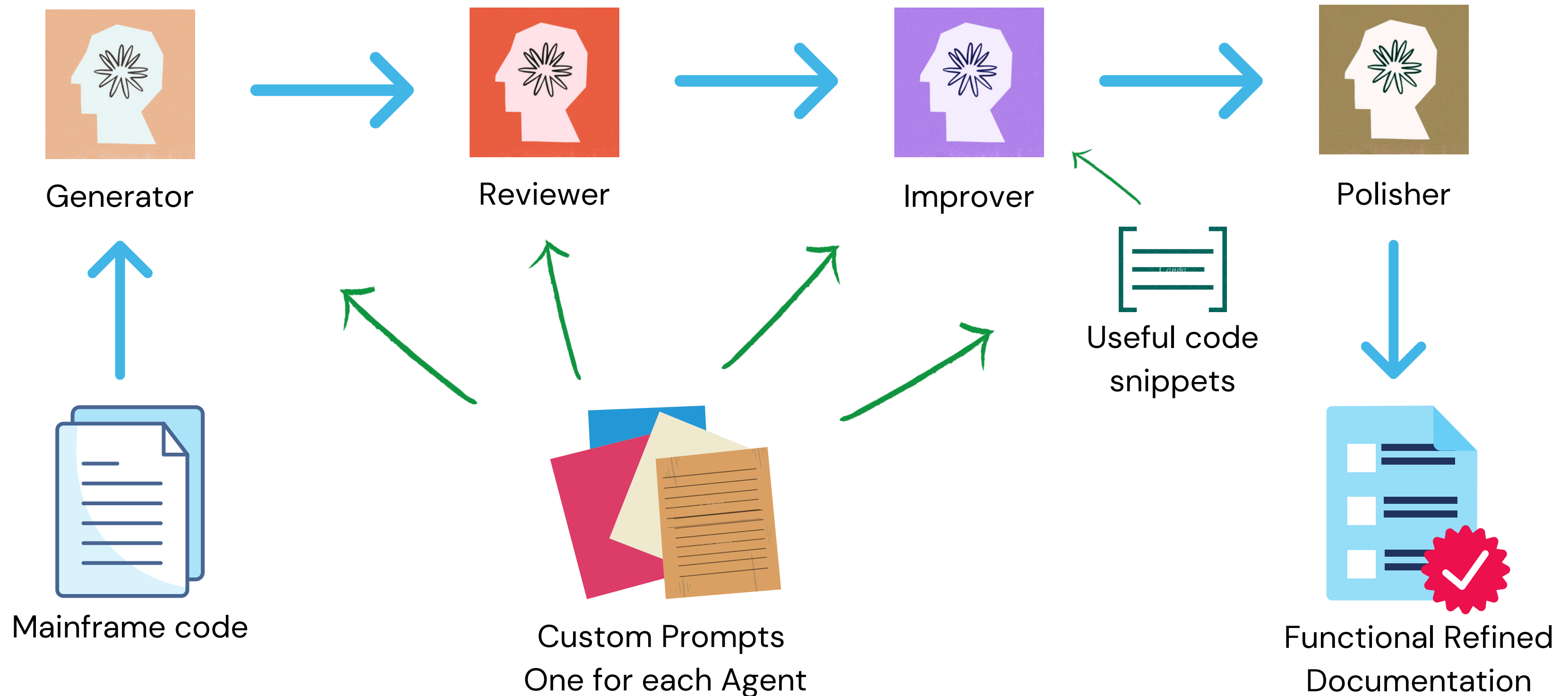


Chain Approach



- Chains are **summarization methods** used to overcome CW limitations of open-source models and have a **scalable solution** that could be applied to any code received as input.
- MapReduce works by dividing the document into smaller **chunks** that fit within the model's context window. Each chunk is processed independently and **partial results** are generated. These partial results are then **aggregated** to form the final output.

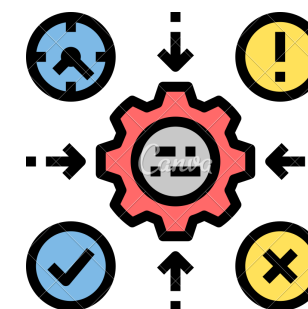
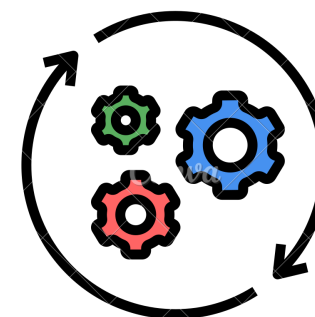
Multi-agent Approach



Prompt Engineering

Techniques Implemented

- Zero-shot prompting
- **Few-shot prompting:** Provide examples of document sections, structure, formatting style, diagrams.
- **Role prompting:** Guide model's response by assigning it a specific role or persona.
- **Prompt Chaining** 
- **Automatic Prompt Engineering:** Create tailored prompts thanks to other LLMs.
- **Context Enhancement:** Add company's specific context: nomenclature, system operation, general context about ES, etc.



Some Results

```
IDENTIFICATION DIVISION.  
PROGRAM-ID. WSOC83.  
  
ENVIRONMENT DIVISION.  
CONFIGURATION SECTION.  
SPECIAL-NAMES.  
    DECIMAL-POINT IS COMMA  
    CLASS NUMERICA IS '0' THRU '9'  
    CLASS ALFABETICA IS 'A' THRU 'Z'  
    CLASS ALFANUM IS 'ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789'.  
  
DATA DIVISION.  
WORKING-STORAGE SECTION.  
    COPY K2SIWRKC.  
    COPY WORKERR.  
  
* COPY TESTATA MESSAGGIO  
2 references  
01 WS-MSG-TEST.  
    COPY WSC01I.  
  
* COPY BODY MESSAGGIO  
2 references  
01 WS-MSG-CORP.  
    COPY WSC83I.  
  
77 FILLER  
77 W-NOME-MODULO  
77 W-CODA-TD  
    2 references  
88 NO-FINE-CODA-TD  
    3 references  
88 FINE-CODA-TD  
77 W-LETTURA-TD  
    3 references  
88 LETTURA-TD-DONE  
77 MTAUT320  
01 AREA-MTAUT320.  
    05 R-YNAZTIT.  
        10 R-YNAZIONE  
        10 R-YSPECIE
```

```
PIC X(16) VALUE  
    'INIZIO WORKING->'.  
PIC X(08) VALUE SPACES.  
PIC 9(01) VALUE ZERO.  
  
    VALUE ZERO.  
  
    VALUE 1.  
PIC 9(01) VALUE ZERO.  
  
    VALUE 1.  
PIC X(08) VALUE 'MTAUT320'.  
  
PIC X(02).  
PIC X(09).
```

Functional document

UTIP0060 - Date Conversion Utility

Table of Contents

- Functional Overview
- Data Structures
- Program Flow
- Error Handling and Logging
- Dependencies
- Database Operations
- Transactional Behavior
- Testing Recommendations

Functional Overview

The COBOL program UTIP0060 is a utility module designed to convert dates between two different formats: Julian date (YYDDD) and Gregorian date (DDMMYY). This program is likely part of the Euronext Securities Milan (ESM) information system, supporting various date-related operations across the platform.

The program performs the following key functions:

- Accepts a Julian date (YYDDD) as input
- Determines if the year is a leap year
- Calculates the corresponding Gregorian date (DDMMYY)
- Handles error conditions for invalid input dates

This utility ensures consistent date handling across the ESM system, which is crucial for accurate record-keeping, transaction processing, and reporting in the context of a central securities depository.

Data Structures

The program uses several key data structures:

- Input/Output Parameters:
 - DATA-YYDDD : Input Julian date (YYDDD format)
 - DATA-GGMAAA : Output Gregorian date (DDMMYY format)
- Working Storage Variables:
 - DATAS : 5-digit numeric field for date manipulation
 - COMODOA and COMODOB : Temporary variables for calculations
 - GIORNI-PER-MESE : Table of days in each month
 - IND : Index for month iteration

Field Name	Description	Format
DATA-YYDDD	Input Julian date	S9(5) COMP-3
DATA-GGMAAA	Output Gregorian date	Group item
DATA-GG	Day of month	99
DATA-MM	Month	99
DATA-AA	Year	99
DATAS	Working date field	9(5)
COMODOA	Temporary calculation field	S9(7) COMP-3
COMODOB	Temporary calculation field	S9(7) COMP-3
GIORNI-PER-MESE	Table of days per month	Group item
IND	Month index	9999 COMP

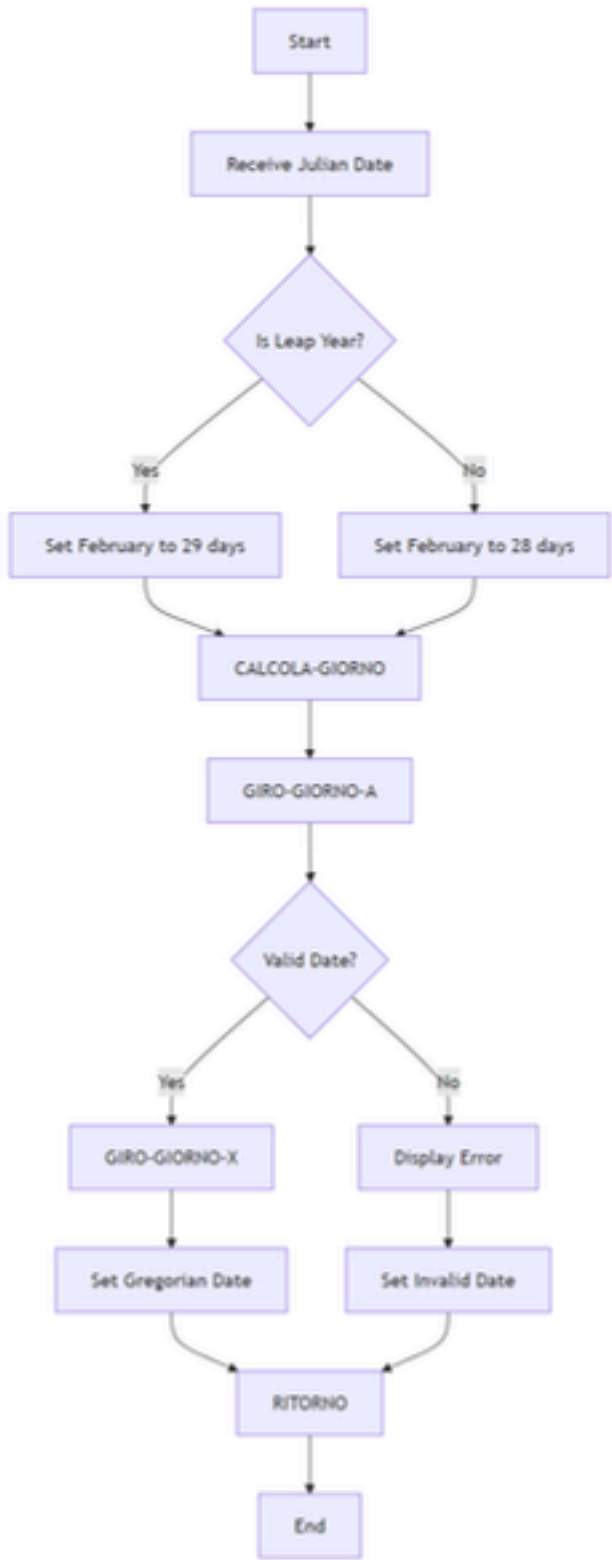
Program Flow

The main processing flow of UTIP0060 is as follows:

- Receive the Julian date (YYDDD) as input
- Determine if the year is a leap year
 - If yes, set February to 29 days
 - If no, set February to 28 days
- Calculate the corresponding Gregorian date (DDMMYY)
- If the calculation is successful, return the Gregorian date
- If an error occurs, display an error message and set an invalid date

Key paragraphs:

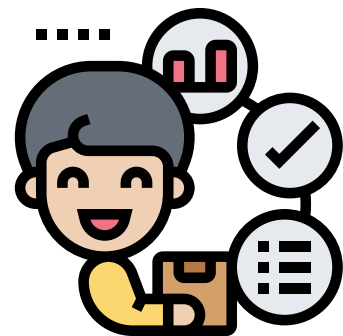
- CALCOLA-GIORNO : Main routine for date calculation
- GIRO-GIORNO-A : Loop to determine the month and day
- GIRO-GIORNO-X : Finalize the date calculation
- RITORNO : Return point for the program



Evaluation Framework

Human Evaluation

- Involve **three expert developers** in the considered programming language.
- Analyze and assess the output according to four criteria: **completeness, clarity, consistency, and accessibility.**
- **Expensive** in terms of both time and resources used.



LLM evaluation

- Use **Claude 3.5 Sonnet** to analyze and assess the output based on same criteria and scores of human evaluation.
- Both **code and documentation** are provided.
- Requires **ad hoc prompt**.
- Free for few outputs, but becomes costly in a scalable solution.



Automated Metrics

- **ROUGE** (-1, -2, -L) and **BERTScore**.
- Used to **compare open-source and closed-source** models' general understanding of the program.
- Applied to **documentation overview**.



Time and Cost

Results and Comparison

Approach	Model	LLM Evaluation	Human Eval. (Avg.)	Time	Cost
Direct	Closed-source	12/12	10.4/12	<1min	\$2-5
Direct	Open-source	9/12	9.3/12	5min	Free
Chain	Open-source	8/12	8.6/12	50min	Free
Multi-agent	Closed-source	12/12	10.8/12	1min	\$5
Multi-agent	Open-source	12/12	10.4/12	10min	Free

(The scores obtained for each evaluation criterion are summed up and presented as an overall score on a 12-point scale.)

Approach	ROUGE-1	ROUGE-2	ROUGE-L	BERTScore
Direct	0.40	0.10	0.27	0.62
Chain	0.36	0.05	0.21	0.63
Multi-agent	0.52	0.25	0.38	0.68

- **Multi-agent approach** with Claude obtains the best scores both in human and LLM evaluation, while **Direct approach** has slightly inferior scores.
- **Open-source** alternatives show **promising potential**.
- LLM evaluations generally **align** with human assessments. Tendency for the LLM to overestimate the quality of outputs.
- Every open-source approach, even not capturing the exact phrasing in terms of n-grams, preserves the **overall semantic content** effectively.

Further Implementations



Documentation enhancement

- Not only comprehensive analyses of legacy programs, but also **strategic recommendations** for potential rewriting or decommissioning of applications and functionalities.
- Side task already explored with **promising results**.



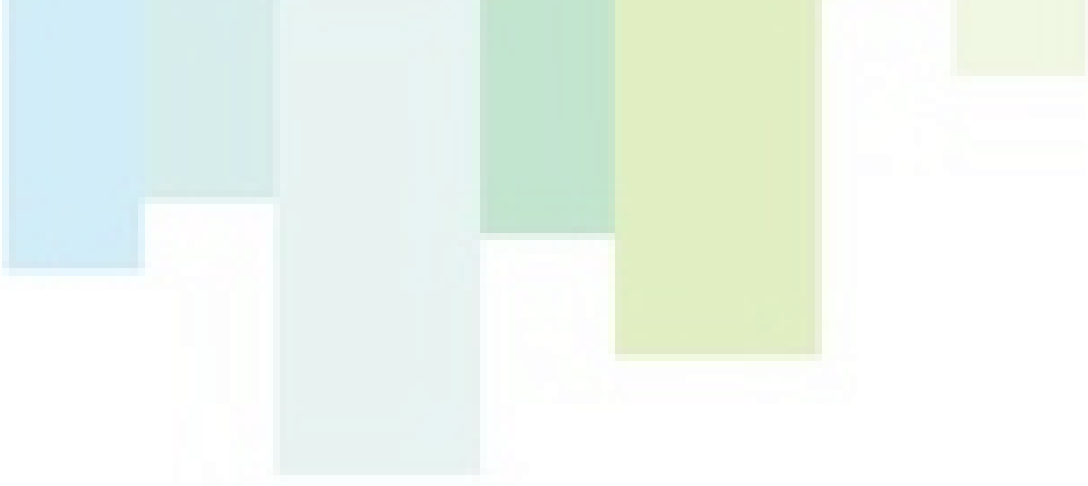
RAG

- Establish a **knowledge source** comprised of both the code and the generated documentation
- LLM, functioning as a **chatbot**, which could access to the knowledge and answer specific user questions about legacy programs and applications.



Fine-tuning

- Fine-tune open-source models to have **ad hoc LLMs** for various tasks in Euronext.



Conclusion



- Significant step towards **automating** the process of legacy system **sustainability** and **modernization**. Potentially saving hundreds of work hours, reducing inefficiency and operational costs.
- Proved the **viability** and **effectiveness** of generative AI in producing functional documentation for legacy mainframe applications.
- Built tailored and efficient **prompts**.
- Assessed the multi-agent approach, using state-of-the-art LLMs, as the best approach for **high quality documentation**, closely followed by the direct approach.
- Validated the **feasibility of using open-source** models for the task, even if with inferior results.
- Extended the experiments **beyond COBOL** programs to include **PL/1** and **EGL**, with comparable output quality across these programming languages.

Thanks for Your Attention



UNIVERSITÀ DEGLI STUDI DI MILANO

